

Accelerating Storage-based Training for Graph Neural Networks

Myung-Hwan Jang
Hanyang University
Seoul, Republic of Korea
sugichiin@hanyang.ac.kr

Jeong-Min Park
Hanyang University
Seoul, Republic of Korea
jmpark96@hanyang.ac.kr

Yunyong Ko
Chung-Ang University
Seoul, Republic of Korea
yyko@cau.ac.kr

Sang-Wook Kim*
Hanyang University
Seoul, Republic of Korea
wook@hanyang.ac.kr

Abstract

Graph neural networks (GNNs) have achieved breakthroughs in various real-world downstream tasks due to their powerful expressiveness. As the scale of real-world graphs has been continuously growing, a *storage-based approach to GNN training* has been studied, which leverages external storage (e.g., NVMe SSDs) to handle such web-scale graphs on a single machine. Although such storage-based GNN training methods have shown promising potential in large-scale GNN training, we observed that they suffer from a severe bottleneck in data preparation since they overlook a critical challenge: *how to handle a large number of small storage I/Os*. To address the challenge, in this paper, we propose a novel storage-based GNN training framework, named AGNES, that employs a method of *block-wise storage I/O processing* to fully utilize the I/O bandwidth of high-performance storage devices. Moreover, to further enhance the efficiency of each storage I/O, AGNES employs a simple yet effective strategy, *hyperbatch-based processing* based on the characteristics of real-world graphs. Comprehensive experiments on five real-world graphs reveal that AGNES consistently outperforms four state-of-the-art methods, up to 4.1× faster than the best competitor.

CCS Concepts

• Information systems → Data management systems; • Computing methodologies → Machine learning.

Keywords

Graph neural networks; Storage-based GNN training

ACM Reference Format:

Myung-Hwan Jang, Jeong-Min Park, Yunyong Ko, and Sang-Wook Kim. 2026. Accelerating Storage-based Training for Graph Neural Networks. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3770854.3780309>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18112452> (<https://github.com/Bigdasgit/agnes-kdd26>).

1 Introduction

Graphs are prevalent in many applications [5, 7, 9, 14, 25, 41] to represent a variety of real-world networks, such as social networks and

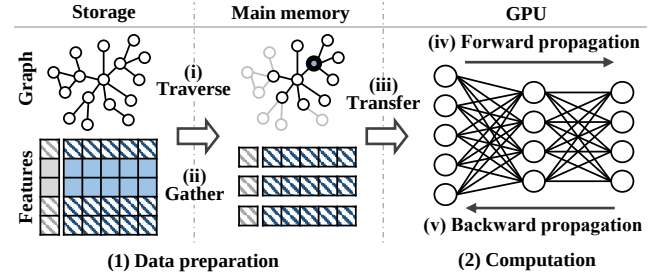


Figure 1: Overview of storage-based GNN training.

web, where objects and their relationships are modeled as nodes and edges, respectively. Recently, *graph neural networks* (GNNs), a class of deep neural networks specially designed to learn such graph-structured data, have achieved breakthroughs in various downstream tasks, including node classification [24, 43], link prediction [20, 33–36], and community detection [3, 19, 38].

Although existing works have designed model architectures to learn the structural information of graphs by considering not only *node features* but also *graph topology* [1, 16, 18, 23, 37], they have a *simple assumption*: the entire input data, including node features and graph topology, *reside in the GPU or main memory* during GNN training [2, 30, 31, 39]. However, as the scale of real-world graphs has been continuously growing, this assumption is not practical anymore: the size of real-world graphs often exceeds the capacity of GPU memory (e.g., 80GB for an NVIDIA H100) or even that of main memory in a single machine (e.g., 256GB). For instance, training 3-layer GAT [28] on the yahoo-web graph [32], which consists of 1.4B nodes and 6.6B edges, requires about 1.5TB, including node features, graph topology, and intermediate results.

To address this challenge, a *storage-based approach to GNN training* has been studied [8, 22, 26, 29], which leverages recent high-performance *external storage devices* (e.g., NVMe SSDs) [8, 22, 29]. This approach stores the entire graph topology and node features in external storage and loads some *parts* into the main memory from storage only when required for GNN training. The storage-based GNN training is two-fold as illustrated in Figure 1:

(1) **Data preparation**: it (i) traverses the graph stored in storage (by loading it into main memory) to find the neighboring nodes of *target nodes* necessary for training, (ii) gathers their associated features stored in storage for training in main memory, and (iii) transfers both into the GPU.

(2) **Computation**: it performs (iv) forward propagation (i.e., prediction) and (v) backward propagation (i.e., loss and gradient computations) over the transferred data in the GPU.

Although the advanced computational power of modern GPUs has accelerated the computation stage significantly, the data preparation stage could be a significant bottleneck in the entire process of the storage-based GNN training as it can incur a large amount of

*Corresponding author.



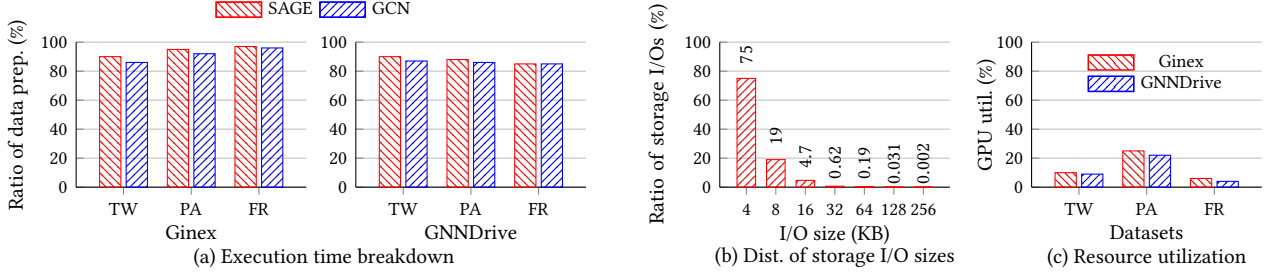


Figure 2: Breakdown of the execution time of state-of-the-art GNN training methods (Ginex [22] and GNNDrive [8]).

I/Os between storage and main memory (simply storage I/Os, hereafter). Existing works [8, 22, 26, 29] have focused on improving the data preparation stage, thus showing its promising potential.

Despite their success, we observed that there is still a large room for further improvement in storage-based GNN training. We conducted a preliminary experiment to analyze the ratio of the time for the data preparation stage to the total execution time in Ginex [22] and GNNDrive [8], state-of-the-art methods for storage-based GNN training. Specifically, we trained two GNN models, i.e., GCN [13] and GraphSAGE [4] (SAGE in short), on three real-world graph datasets – twitter-2010 (TW) [11], ogbn-papers100M (PA) [6], and com-friendster (FR) [11]. As shown in Figure 2(a), *the data preparation stage dominates the entire training process* (i.e., up to 96% of the total execution time). For in-depth analysis, we also measured the size of each individual I/O that occurs during training. Figure 2(b) shows the distribution of storage I/Os’ sizes, where *a large number of storage I/Os are small*, while only a few I/Os are very large. Such a large number of small I/Os leads to significant degradation of the utilization of computing resources (e.g., GPU utilization) in the GNN training as shown in Figure 2(c).

We posit that this phenomenon arises because real-world graphs tend to have a *power-law degree distribution* [15], meaning that the majority of nodes have only a few edges (i.e., neighbors) while a small number of nodes have a huge number of edges. That is, the number of neighboring nodes required for GNN training is highly likely to be very small in most cases. Existing storage-based GNN training methods [8, 22, 26, 29], however, have overlooked this important characteristic. They focus only on *how to increase the possibility of reusing cached data in main memory* (i.e., cache hit ratio) and simply read a few nodes from storage whenever they are required for GNN training, thereby *generating a significant number of small storage I/Os*. For example, Sheng et al. [26] aims to enhance the locality of sampled nodes for better cache hit ratio by partitioning the entire graph and selecting target nodes within the same partition. These approaches, however, do not address the challenge of handling a large number of small I/Os fundamentally, which still remains under-explored.

We may tackle this challenge by merging small storage I/Os and processing them together. However, simply increasing the I/O unit size is not an optimal way to solve the problem since a large amount of unnecessary data (i.e., nodes/edge unrelated to target nodes) can be included in each I/O, which can waste the main memory space. **Our work.** In this paper, to address the aforementioned fundamental challenge of storage-based GNN training, we propose a novel framework, named as **AGNES**, that has a 3-layer architecture with (i) storage, (ii) in-memory, and (iii) operation layers where each

layer interacts closely with the others for efficiently handling storage I/Os. We present a method of *block-wise storage I/O processing* with a *novel data layout* to reduce the number of small storage I/Os, thereby fully utilizing the power of high-performance storage devices—the I/O bandwidth (I/O-BW). Moreover, to further improve the efficiency of each storage I/O (i.e., the cache hit ratio), we propose a simple yet effective strategy based on the characteristics of real-world graphs: *hyperbatch-based processing*, which carefully collects the data required for GNN training within blocks as much as possible and process them all at once in each iteration.

Contributions. The main contributions of this work are as follows.

- **Observations:** We observe that existing works have overlooked a critical yet under-explored challenge of storage-based GNN training: *how to handle a large number of small storage I/Os*.
- **Framework:** We propose a novel framework for storage-based GNN training, AGNES, that effectively addresses the challenge by employing block-wise storage I/O processing and hyperbatch-based processing.
- **Evaluation:** Comprehensive experiments using five real-world graphs reveal that AGNES significantly outperforms state-of-the-art storage-based GNN training methods. Specifically, AGNES finishes training by up to $4.1\times$ faster, while achieving the utilization of I/O-BW by up to $4.5\times$ greater than the best competitor.

2 Related Works

In this section, we review existing GNN training approaches and explain their relation to our work.

Storage-based approaches. Recently, storage-based GNN training approaches, our main focus, have been studied, which leverage external storage on a single machine for large-scale GNN training [8, 22, 26, 29]. These approaches store the entire graph topology and node features in the external storage and load only the parts required for GNN training into main memory. Ginex [22], the state-of-the-art storage-based GNN training method, employs a caching mechanism for node feature vectors, which addresses I/O congestion issues effectively, thereby successfully handling billion-scale graph datasets on a single machine. MariusGNN [29] partitions the graph into multiple partitions, buffering the partitions in main memory and reusing sampled results to mitigate storage I/O bottlenecks. It also adopts a data structure to minimize the redundancy of multi-hop sampling and the two-level minibatch replacement policy for disk-based training. GNNDrive [8] employs buffer management across different stages that support the sample stage to relieve memory contention. It also uses an asynchronous feature extraction to address memory usage issues and alleviate storage I/O

bottlenecks. OUTRE [26] employs partition-based batch construction and historical embedding to reduce neighborhood redundancy and temporal redundancy in sampling-based GNN training.

Although the existing storage-based approaches have shown promising potential for large-scale GNN training on a single machine, they still suffer from a significant bottleneck in the data preparation stage (as shown in Figure 2) since they overlook the natural but critical challenge. To the best of our knowledge, this is the first work to address the challenge in detail.

Other approaches. In addition to the storage-based approach, memory-based and distributed-system-based approaches have been studied. Memory-based approaches [2, 17, 21, 27, 30] store graph data or node features in the main memory to handle large-scale graphs that exceed the GPU memory size. For instance, PyG [2] employs a method of utilizing both CPU and GPU to improve the training speed of GNN models. DGL [30] adopts a zero-copy approach for fast data transfer from main memory to GPU. PaGraph [17] addresses data transfer bottleneck between CPU and GPU by caching frequently accessed high out-degree nodes in GPU memory. On the other hand, distributed-system-based approaches [40, 42, 44] leverage abundant computing power and memory capacity of distributed systems for training GNN models on very large graphs that even exceed the capacity of a single machine. AliGraph [44] adopts a method of caching node data locally on each machine to reduce network communication costs. DistDGL [40] splits a given graph using a min-cut partitioning algorithm to not only reduce network communication costs but also balance the graph partitions and minibatches generated from each partition. DistDGLv2 [42] improves DistDGL by using a multi-level partitioning algorithm and an asynchronous minibatch generation pipeline.

However, memory-based approaches cannot handle large-scale graphs that exceed the capacity of the main memory (i.e., less scalable), and distributed-system-based approaches require a substantial amount of *inter-machine communication* overhead to aggregate the results from multiple machines and *costs and efforts* to maintain high-performance distributed systems (i.e., less efficient and costly).

3 Proposed Framework: AGNES

In this section, we propose a novel framework for storage-based GNN training, named **Accelerating storage-based training for Graph Neural networks** (AGNES).

3.1 Preliminaries

The notations used in this paper are described in Table 1.

Graph neural networks (GNNs). GNNs aim to represent the embedding vectors of nodes based on a graph structure via a message passing mechanism [4, 13]. More specifically, each layer of a GNN consists of two steps: aggregation and update. In the aggregation step, for each node, the embedding vectors of its in-neighbors are aggregated into the embedding of the target node. In the update step, the aggregated embeddings are passed through a fully connected layer with a nonlinear function. The update of an embedding $\mathbf{h}_v \in \mathbb{R}^d$ can be represented as (1):

$$\mathbf{h}_v^{(i+1)} = \psi(\phi(\mathbf{h}_{v'}^{(i)} | v' \in N(v), \mathbf{h}_v^{(i)})) \quad (1)$$

where $N(v)$ denotes the set of the neighboring nodes of node v , $\psi(\cdot)$ and $\phi(\cdot)$ are aggregation and update functions, respectively. By

Table 1: Notations and their descriptions

Notation	Description
$\mathbf{h}_v^i$	an embedding of a node v from i -th layer
$\psi(\cdot), \phi(\cdot)$	aggregation and update functions
$N(v)$	a set of the neighboring nodes of a node v
B_g, B_f	topology and features of input graph
T_{buf}^g, T_{buf}^f	buffer index tables for topology B_g and features B_f
T_{obj}^g	object index table for topology B_g
C_f	feature cache for features B_f
T_{ch}^f	cache index table for features in feature cache C_f

stacking multiple layers, a GNN model can reflect the information of k -hop neighboring nodes of a target node into its embeddings, where each layer is responsible for aggregating and updating the information of neighboring nodes from the corresponding hop.

Minibatch training for GNNs. Meanwhile, storing a real-world graph with its node features often requires more than hundreds of gigabytes (GB) or even tens of terabytes (TB), exceeding the capacity of main memory. To handle such a large graph on a single machine, a storage-based GNN training method stores the entire graph in external storage (e.g., NVMe SSDs) and processes only a subset of nodes (i.e., a *minibatch*) from the entire graph, which can be loaded into GPU memory, at each iteration [22, 29].

Even in minibatch training of GNNs, however, collecting all k -hop neighboring nodes of a target node and their feature vectors may require a large amount of memory [22, 29]. To address this memory issue, existing storage-based methods employ a simple strategy that (i) randomly samples only a subset of neighboring nodes related to a target node and (ii) uses them to update its embedding in GNN training [8, 22, 29].

Two stages of storage-based GNN training. Figure 1 shows an overview of the two main stages in storage-based GNN training: (1) data preparation and (2) computation. In storage-based GNN training, CPU and GPU collaborate interactively to efficiently perform large-scale GNN training [2, 8, 22, 29], where the CPU is in charge of (1) the data preparation and the GPU is in charge of (2) the computation. In the data preparation stage, the CPU (i) traverses a graph to find the neighboring nodes required for training, (ii) gathers their feature vectors, and (iii) transfers them to the GPU. In the computation stage, the GPU performs (iv) forward and (v) backward propagation (i.e., gradient computations) for each minibatch.

3.2 Architecture of AGNES

AGNES has a 3-layer architecture with storage, in-memory, and operation layers where each layer interacts closely with the others for efficient management. Figure 3 shows the overview of AGNES.

(1) Storage layer. This layer manages storage space and manages graph topology and node features stored in the storage. Specifically, it divides and stores the graph topology and feature vectors into multiple *blocks* (fixed-size storage I/O unit). There are two types of blocks: (1) graph block and (2) feature block. (1) A graph block contains multiple objects (i.e., multiple nodes and their related edges). (2) A feature block contains multiple feature vectors. If an object exceeds the size of a single block (i.e., a node with a large number of edges), the object is split across multiple blocks. This layer is in charge of handling I/O requests from the in-memory layer, where all I/Os are processed in a block-wise manner.

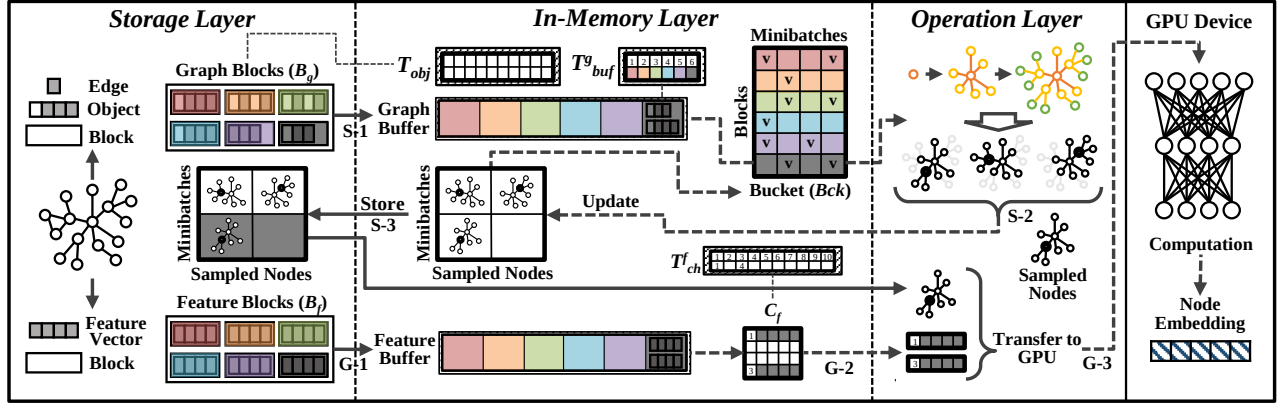


Figure 3: Overview of AGNES with the 3-layer architecture (Storage, In-memory, and Operation layers).

To enhance the efficiency of block-wise storage I/O processing, we employ an efficient data layout by following [9, 10]. The key idea is to place the data accessed together during a graph algorithm in the same (adjacent) blocks. Since AGNES stores objects (each having a node and its edges) in blocks in the ascending order of node IDs, we assign consecutive node IDs to the nodes likely to be accessed together at the same or adjacent iteration(s) by a graph algorithm. As a result, not only the number of accessed blocks is reduced but also the degree of sequential accesses of blocks increases, thereby enhancing the efficiency of block-wise storage I/O processing.

(2) In-memory layer. This layer manages the buffers in the main memory. Specifically, this layer is in charge of the following three tasks: (1) loading the required blocks into main memory; (2) storing the sampled results from the operation layer in the storage; and (3) gathering minibatch workloads required for GNN computation. This layer defines the following components:

- Graph buffer and feature buffer: storing the graph blocks and the feature blocks loaded from storage, respectively.
- Buffer index tables (T_{buf}^g and T_{buf}^f): indicating the address where the blocks are located in the graph buffer or feature buffer.
- Object index table (T_{obj}^g): mapping each block to the objects in storage, where each column corresponds to the objects stored in the corresponding block, indexed by their node IDs.
- Feature cache (C_f): storing the node features required for processing each minibatch.
- Cache index table (T_{ch}^f): tracking the location of each node's feature in the feature cache.

To efficiently use main memory, we only store the first and last object indices for each block in the object index table, sorted in ascending order by node IDs. The object index table is always pinned in the main memory to quickly identify the location of required blocks in storage. Since this table occupies less than 0.01% of the size of the original graph, it does not affect the overall performance.

(3) Operation layer. This layer is responsible for performing CPU computations involved in data preparation using the data in B_g and B_f . The sampling process consists of the following steps: (S-1) reading the required objects from B_g by referring to T_{buf}^g , (S-2) performing traversal and sampling neighboring nodes, and (S-3) updating the sampled nodes (target nodes and their k -hop neighbors)

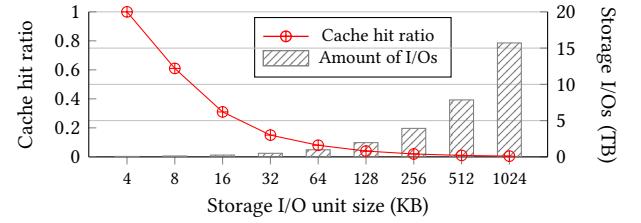


Figure 4: Cache hit ratio and amount of storage I/Os of Ginex [22] with varying storage I/O unit sizes.

and storing them in storage. Then, the gathering and transferring process consists of the following steps: (G-1) reading the associated feature vectors of the sampled k -hop neighboring nodes from B_f by referring to T_{ch}^f , (G-2) collecting the associated feature vectors into a contiguous memory space, and (G-3) transferring the sampled nodes and their feature vectors to the GPU, which performs GNN computations. This process is repeated until all minibatches are processed (e.g., one epoch).

3.3 Hyperbatch-based Processing

Motivation. As mentioned in Section 1, handling a large number of small I/Os is a key challenge of storage-based GNN training. We may tackle this challenge by increasing the size of a storage I/O unit and processing multiple units together, thereby improving the resource utilization. However, the simple increase may lead to low efficiency of each storage I/O since a large amount of unnecessary data can be included in each I/O, which can waste the main memory space. In order to evaluate the effect of increasing the size of a storage I/O unit, we measure the amount of storage I/Os and relative data reuse of Ginex [22] with varying unit sizes on the PA dataset. Figure 4 shows that, as the size of storage I/O unit increases, the total amount of storage I/Os (bar) grows, surpassing even 15 terabytes (TB), and the cache hit ratio decreases to below 0.06%. These results imply that simply increasing the storage I/O unit size is not a solution to this challenge. In addition, due to the limited size of the main memory, the loaded data can be replaced with other data even though they are necessary for GNN training later. In this case, the replaced data must be reloaded *multiple times* from storage, which can incur an unnecessarily large amount of storage I/Os.

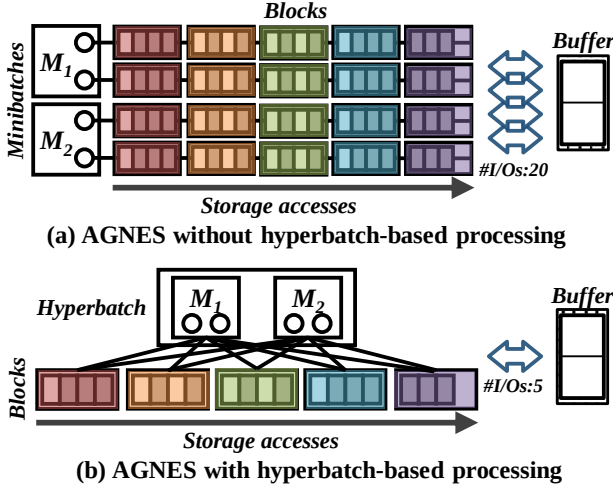


Figure 5: Effect of hyperbatch-based processing. It significantly reduces storage I/Os (e.g., 20 I/Os $\rightarrow$ 5 I/Os).

Key idea. From this motivation, we present a simple yet effective strategy to improve the efficiency of each storage I/O: *hyperbatch-based processing*. The idea behind this method is as follows: if we process a single target node (minibatch) independently, it requires additional storage I/Os since many other nodes in the loaded block are not needed for the current target node. Instead, processing nodes within the same block and reusing them across different target nodes (minibatches) together requires only a single block-wise storage I/O for each block. In other words, our hyperbatch-based processing extends its processing scope from the target nodes in a single minibatch to those in multiple minibatches (we call it ‘hyperbatch’ hereafter).

Example 1 (w/o hyperbatch-based processing). Figure 5(a)-(b) show an example of the data preparation process for four target nodes in two minibatches and five blocks with a buffer space of two blocks. Each circle represents a target node, colored boxes represent the blocks containing the data (e.g., adjacency lists or feature vectors). Here, blocks contain multiple data required by all target nodes. Figure 5(a) illustrates the case when storage I/Os are performed from the perspective of target nodes (minibatches). Whenever each block (e.g., a red box) containing neighboring nodes of the target node is processed, the block may be replaced with other blocks (e.g., blue/purple boxes) required for processing other target nodes (minibatches). In this case, the replaced block should be *reloaded* from storage, resulting in a large number of storage I/Os (e.g., 20 storage I/Os).

Example 2 (w/ hyperbatch-based processing). Figure 5(b) illustrates the case when storage I/Os are performed from the perspective of blocks; after a block (e.g., a red box) required for GNN training is loaded in main memory, *hyperbatch-based processing* processes neighboring nodes of multiple target nodes within the same minibatch and target nodes of a hyperbatch within a single iteration. Consequently, this simple strategy significantly reduces additional storage I/Os (e.g., 5 storage I/Os).

We will verify the effectiveness of hyperbatch-based processing on the performance of AGNES in Section 4.3.

3.4 Performance Consideration

In this section, we describe four key design considerations that affect the performance of AGNES.¹

(1) Sampling process. This process finds k -hop neighboring nodes of target nodes to be updated by a GNN model. Each target node goes through multiple iterations corresponding to the number of hops (i.e., the number of GNN layers). As the layer gets deeper, the number of neighbors grows significantly. Since hyperbatch-based processing handles multiple target nodes in multiple minibatches at once, it is likely that the same blocks are required in consecutive training iterations. To further enhance the efficiency of each storage I/O, AGNES uses dynamic caching based on an LRU mechanism, also adopted in [5, 9], during the sampling stage, to pin graph blocks already in the graph buffer (e.g., the blocks processed in previous iterations) to prevent them from being replaced until they are completely processed in the current iteration. AGNES unpins these blocks after they are completely processed.

(2) Gathering process. This process collects the sampled k -hop neighboring nodes of the target nodes and their feature vectors for transfer to the GPU. The required feature vectors are moved to a *contiguous* memory space in one iteration. Compared to graph topology, feature vectors require much larger storage space than graph topology [21, 27]. To efficiently use main memory space, AGNES counts the number of accesses to each feature vector and maintains only feature vectors whose access counts exceed a certain threshold, in a feature cache in main memory. While the others (i.e., infrequently accessed feature vectors) are written back to storage at each minibatch and reloaded when they are required.

(3) Node identification. To manage the sampled nodes in different minibatches, we define a *bucket*, Bck , which is a matrix containing the information of sampled nodes (i.e., their block IDs and minibatch IDs). Bck has rows and columns corresponding to the number of blocks and minibatches in a hyperbatch (i.e., the size of the hyperbatch), respectively. Each cell of Bck , $Bck_{i,j}$, includes the nodes to be processed in the corresponding minibatch within a specific block. Thus, AGNES identifies the nodes to be processed efficiently by scanning a row of the matrix, $Bck_{i,:}$. Specifically, given target nodes to be processed, AGNES first (1) finds the index(es) of the block(s) containing the target nodes by referring to the object index table T_{obj}^g ; (2) loads them into the main memory; and (3) identifies the target nodes and their minibatch IDs by scanning $Bck_{i,:}$, the i -th row of the bucket matrix that corresponds to the i -th block.

(4) Asynchronous I/O. AGNES continuously loads parts of graph topology and feature vectors from storage to main memory, which are much slower than in-memory data transfers. To achieve higher I/O-BW from *more-frequent I/O requests* for the blocks to be processed, AGNES adopts *asynchronous I/O* processing. After a thread issues an I/O request to the storage, the thread does not wait for the completion of the I/O in an idle state but rather tries to take over other tasks required to be processed. This simple strategy could hide the costly I/O time within the overhead of other tasks, thereby accelerating the process of data preparation.

¹All implementation details are available at <https://github.com/Bigdasgit/agnes-kdd26>.

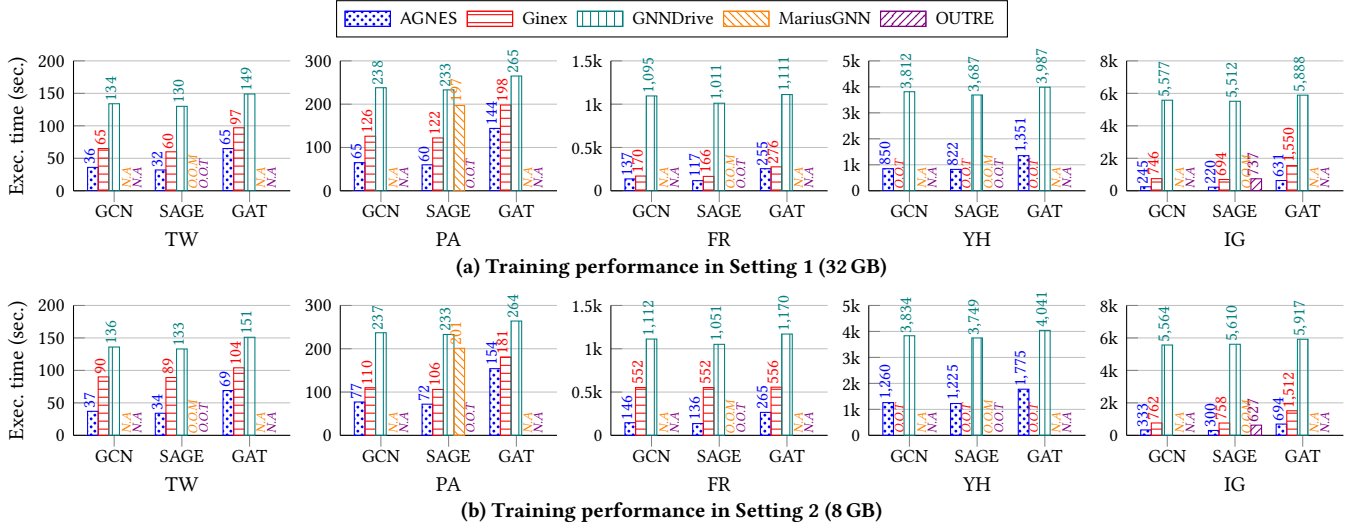


Figure 6: Comparison with storage-based training methods: AGNES consistently outperforms *all* the state-of-the-art storage-based methods. AGNES performs GNN training up to 4.1× and 3.4× faster than Ginex and OUTRE, respectively.

4.2 EQ1. Training Performance

We compare the training performance of AGNES with those of four state-of-the-art storage-based and one distributed GNN training methods: Ginex [22], MariusGNN [29], GNNDrive [8], OUTRE [26], and DistDGL [40]. We consider two different buffer sizes for graph topology and node features: (1) Setting 1 (32 GB): this setting reflects a more-practical configuration commonly adopted in existing storage-based GNN training methods [8, 22, 26, 29]. (2) Setting 2 (8 GB): this setting is considered to rigorously evaluate the I/O handling capability of each method under constrained memory conditions, simulating the case where the graph size is significantly larger than main memory (i.e., I/O intensive setting).

Comparison with storage-based training methods. As shown in Figure 6(a)–(b), AGNES consistently outperforms *all* state-of-the-art storage-based GNN training methods across *all* datasets. Specifically, in Setting 1, AGNES achieves up to a 3.1× speedup over the best-performing competitor, Ginex. In Setting 2, AGNES further widens the performance gap, outperforming Ginex up to 4.1×. This result indicates that AGNES can effectively handle storage I/O operations even under constrained memory conditions. The superior performance of AGNES arises from its ability to mitigate a critical bottleneck in existing approaches (i.e., handling a large amount of small storage I/Os). Existing methods issue a large number of small storage I/O requests upon cache misses, which prevents them from fully exploiting the I/O-BW provided by NVMe SSDs. In contrast, although AGNES loads a larger amount of data via block-wise storage I/Os, it completes these I/O operations efficiently by fully utilizing the available NVMe I/O-BW. As a result, the overhead of storage I/O is substantially reduced, leading to a significant improvement in training performance.

Note that N.A indicates a not-available case (e.g., MariusGNN and OUTRE support only the GraphSAGE model), O.O.M denotes an out-of-memory case, as also reported in prior studies [8, 27], and O.O.T represents an out-of-time case, where the entire execution, including preprocessing and training, requires more than 48 hours.

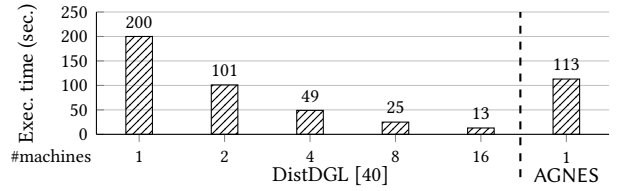


Figure 7: Comparison with a distributed training method: AGNES achieves comparable training performance to a distributed method only with limited computing resources.

Comparison with a distributed training method. In addition, we evaluate AGNES against a distributed GNN training approach, DistDGL [40]. DistDGL was evaluated on a cluster of 16 AWS m5.24xlarge instances, each equipped with 96 vCPUs, 384 GB memory, and interconnected via a 100 Gbps network [40]. Since replicating such a high-end distributed environment is infeasible, we quote the performance results on the PA dataset as reported in [40].

Figure 7 shows that AGNES achieves performance comparable to DistDGL running on two instances, despite being executed on a *single machine with limited computational resources*. Considering that DistDGL eliminates storage I/Os by maintaining the *entire graph in large main memory* across high-end distributed nodes, this result highlights the efficiency of AGNES in enabling large-scale GNN training with substantially lower infrastructure requirements. This advantage arises since AGNES incurs only intra-machine communication overhead (i.e., storage I/Os), which is significantly smaller than the inter-machine communication overhead inherent to distributed systems. Moreover, this comparison demonstrates that carefully optimized storage-based GNN training can narrow the performance gap with distributed training approaches. As a result, AGNES provides a *practical* and *cost-effective* alternative for large-scale GNN training, especially in scenarios where access to expensive distributed clusters is limited or impractical. These findings further validate the scalability of AGNES for large-scale GNN training under realistic resource constraints.

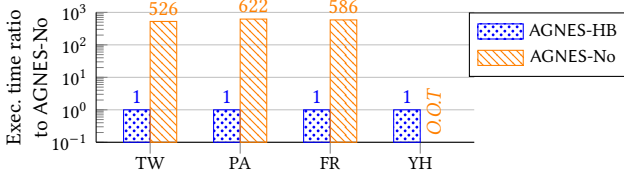


Figure 8: Effect of the hyperbatch-based processing on the performance of AGNES. It significantly improves the training performance of AGNES.

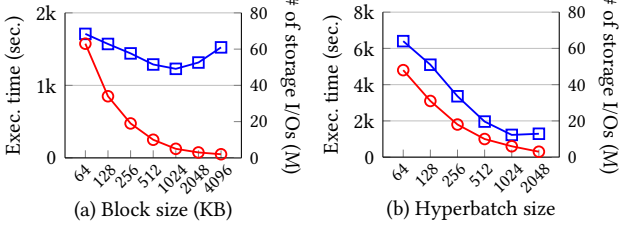


Figure 9: Execution time (blue) and the number of storage I/Os (red) according to block size and hyperbatch size.

4.3 EQ2. Ablation Study

Then, we evaluate the effectiveness of the hyperbatch-based processing. We compare the two following versions of AGNES:

- AGNES-No: AGNES without hyperbatch-based processing.
- AGNES-HB: AGNES with hyperbatch-based processing.

Figure 8 shows the ratio of the execution time of AGNES-No compared to that of AGNES-HB. The results show that our proposed strategy significantly enhances GNN training performance. Specifically, the hyperbatch-based processing improves the execution time of AGNES by up to 622 $\times$ by significantly increasing the efficiency of each storage I/O. This improvement arises because the hyperbatch-based processing effectively eliminates a large number of redundant and small storage I/Os by aggregating them into fewer, larger, and more contiguous I/O requests. As a result, the overhead caused by frequent small storage I/Os can be greatly reduced, allowing AGNES to fully utilize the I/O-BW. These results suggest that mitigating excessive small storage I/Os is as critical as efficiently exploiting in-memory caches, especially in storage-based GNN training methods. Note that O.O.T. denotes an out-of-time case, where execution requires more than 24 hours.

We also conduct experiments to validate the effects of block size and hyperbatch size using the YH dataset, the largest dataset in our experiments. We measure (1) the number of storage I/Os and (2) the total execution time by varying the block size from 64KB to 4096KB and the hyperbatch size from 64 to 2048. Figure 9(a)–(b) shows the results, where the x -axis represents the block/hyperbatch size, and the y -axis represents the total execution time (left) and the total number of storage I/Os (right). First, AGNES achieves the best performance when the block size is 1024KB. Thus, although the number of storage I/Os decreases as the block size increases, the proportion of unnecessary data fetched within each block also increases. Second, AGNES shows the best performance when the hyperbatch size is larger than 1024. This result indicates that, as the hyperbatch size increases, the number of storage I/Os required across multiple minibatches decreases, while the overhead of hyperbatch-based processing gradually increases.

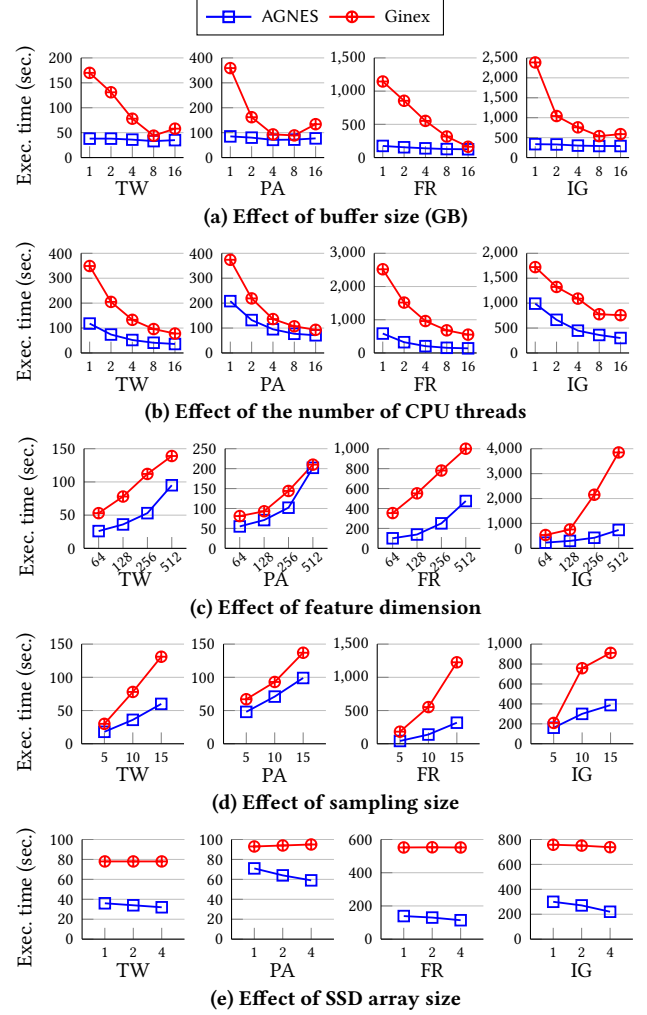


Figure 10: Sensitivity analysis: AGNES (1) consistently outperforms Ginex across various hyperparameters and (2) is less sensitive to hyperparameters than Ginex.

4.4 EQ3. Sensitivity Analysis

We evaluate how sensitive the performance of AGNES is to hyperparameters: (1) buffer size, (2) the number of CPU threads, (3) feature dimension, (4) sampling size, and (5) SSD array size.

(1) Buffer size. Figure 10(a) shows the performance of AGNES and Ginex with varying buffer sizes from 1 GB to 16 GB, where the x -axis denotes the buffer size and the y -axis denotes the execution time. The execution time of Ginex increases rapidly as the buffer size decreases. This result indicates that a smaller buffer size leads to a large number of small storage I/Os, thereby significantly degrading overall performance. In contrast, the performance of AGNES remains stable across different buffer sizes, indicating that AGNES efficiently utilizes data within each block and thus substantially reduces the number of storage I/Os. Notably, even when the buffer size increases to 16 GB for the TW and PA datasets, the execution time of Ginex also increases. This is because Ginex requires a considerable amount of time to load the entire cache into main memory before starting the data preparation process. Therefore,

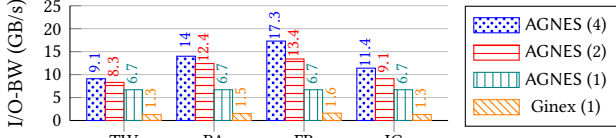


Figure 11: The maximum I/O-BW utilization of AGNES and Ginex according to the number of SSDs.

these results demonstrate that the performance of AGNES is less sensitive to variations in main memory size.

(2) Number of CPU threads. Figure 10(b) shows the performance of AGNES and Ginex with varying numbers of CPU threads from 1 to 16, where the x -axis denotes the number of CPU threads and the y -axis denotes the execution time. The execution time of both methods decreases as the number of threads increases. Notably, AGNES achieves a larger performance improvement than Ginex as the number of threads increases. This result indicates that AGNES utilizes multiple CPU threads more effectively during the data preparation stage, thereby better exploiting parallelism compared to Ginex.

(3) Feature dimension. Figure 10(c) shows the performance of AGNES and Ginex with varying feature dimensions from 64 to 512, where the x -axis denotes the feature dimension and the y -axis does the execution time. The execution time of AGNES increases as the feature dimension increases. Notably, while AGNES is consistently faster than Ginex, the performance improvement is more pronounced at smaller feature dimensions. This is because, when the feature dimension is smaller, AGNES can retrieve more feature vectors with a single block-wise storage I/O. In contrast, Ginex’s small storage I/O, with a minimum size of 4 KB, results in poor data utilization for smaller feature vectors.

(4) Sampling size. Figure 10(d) shows the performance of AGNES and Ginex with varying sampling sizes from 5 to 15, where the x -axis denotes the sampling size per layer and the y -axis does the execution time. The execution time of AGNES increases linearly as the sampling size increases. As the sampling size increases, Ginex experiences a significant increase in small storage I/Os, since more k -hop neighboring nodes are required for GNN training. In contrast, AGNES benefits from the reduced number of block-wise storage I/Os, thereby improving its performance.

(5) SSD array size. Figure 10(e) shows the performance of AGNES and Ginex with varying SSD array sizes from 1 to 4^2 , where the x -axis denotes the SSD array size and the y -axis does the execution time. AGNES reduces the overall execution time by approximately 18% on average and achieves up to a 27% reduction on the IG dataset. On the other hand, the execution time of Ginex remains unchanged even as the SSD array size increases. This is because a large number of small storage I/Os prevents Ginex from fully utilizing the I/O-BW of even a single NVMe SSD. Figure 11 shows the maximum I/O-BW utilization of AGNES and Ginex, where the x -axis denotes datasets and the y -axis does I/O-BW utilization. The results show that AGNES is able to fully utilize the I/O-BW provided by multiple NVMe SSDs by up to 17.3 GB/s.

²We configure RAID0 with Linux mdadm to utilize I/O-BW provided by all NVMe SSDs in parallel.

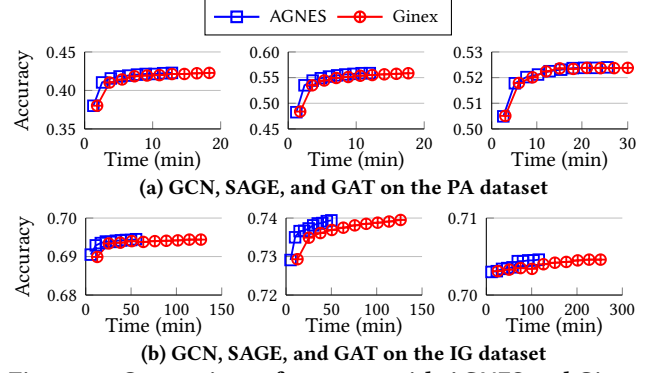


Figure 12: Comparison of accuracy with AGNES and Ginex.

4.5 EQ4. Accuracy

Finally, we evaluate the performance of AGNES in terms of accuracy per training time. We train three GNN models on the PA and IG datasets using AGNES and Ginex for 10 epochs, and measure their node classification accuracy at every epoch. Figure 12 shows the results, where the x -axis denotes the elapsed time and the y -axis does the accuracy. The results show that AGNES achieves the same accuracy as Ginex at every epoch, regardless of the dataset and GNN model. However, AGNES reaches the same accuracy in a shorter amount of time, resulting in a higher accuracy improvement per unit time compared to Ginex. This indicates that AGNES enables faster convergence while preserving model accuracy. Overall, these results demonstrate that AGNES is a more effective solution for large-scale GNN training, as it substantially improves training efficiency without sacrificing accuracy.

5 Conclusions

In this paper, we observe that the existing storage-based GNN training methods suffer from a serious bottleneck of data preparation since they have overlooked a natural yet critical challenge: *how to handle a large number of small storage I/Os*. To address this challenge, we propose a novel storage-based approach to GNN training (AGNES) with the 3-layer architecture to handle storage I/Os efficiently. We identify important issues causing serious performance degradation in storage-based training and propose a simple yet effective strategy, hyperbatch-based processing, that improves the efficiency of storage I/Os based on the unique characteristics of real-world graphs. Through extensive experiments using five web-scale graphs, we demonstrate that AGNES significantly outperforms state-of-the-art storage-based GNN training methods in terms of accelerating large-scale GNN training.

Acknowledgments

This is a joint work between Samsung Electronics Co., Ltd, and Hanyang University (No. IO251222-14891-01); the authors also would like to thank SMRC (Samsung Memory Research Center) for providing the infrastructure for this work. This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2020-II201373 and No. RS-2022-00155586). The work of Yunyong Ko was supported by the National Research Foundation of Korea (NRF) grant, funded by the Korea government (MSIT) (No. RS-2024-00459301).

References

- [1] Ulugbek Ergashev, Eduard Dragut, and Weiyi Meng. 2023. Learning to rank resources with GNN. In *Proceedings of the ACM Web Conference 2023 (WWW)*. ACM, 3247–3256.
- [2] Matthias Fey and Jan Eric Lenssen. 2019. Fast graph representation learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428* (2019).
- [3] Jun Gao, Jiazun Chen, Zhao Li, and Ji Zhang. 2021. ICS-GNN: lightweight interactive community search via graph neural network. *Proceedings of the VLDB Endowment* 14, 6 (2021), 1006–1018.
- [4] Will Hamilton, Zhitaoying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [5] Wook-Shin Han, Sangyeon Lee, Kyungyeol Park, Jeong-Hoon Lee, Min-Soo Kim, Jinha Kim, and Hwanjo Yu. 2013. TurboGraph: A Fast Parallel Graph Engine Handling Billion-Scale Graphs in a Single PC. In *Proceedings of the ACM International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, 77–85.
- [6] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems* 33 (2020), 22118–22133.
- [7] Myung-Hwan Jang, Yunyong Ko, Hyuck-Moo Gwon, Ikhyeon Jo, Yongjun Park, and Sang-Wook Kim. 2023. SAGE: A Storage-Based Approach for Scalable and Efficient Sparse Generalized Matrix-Matrix Multiplication. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 923–933.
- [8] Qisheng Jiang, Lei Jia, and Chundong Wang. 2024. GNNDrive: Reducing Memory Contention and I/O Congestion for Disk-based GNN Training. In *Proceedings of the 53rd International Conference on Parallel Processing (ICPP)*. 650–659.
- [9] Yong-Yeon Jo, Myung-Hwan Jang, Sang-Wook Kim, and Sunju Park. 2019. Real-Graph: A Graph Engine Leveraging The Power-Law Distribution of Real-World Graphs. In *Proceedings of the 2019 World Wide Web Conference (WWW)*. ACM, 807–817.
- [10] Yong-Yeon Jo, Myung-Hwan Jang, Sang-Wook Kim, and Sunju Park. 2021. A Data Layout with Good Data Locality for Single-Machine based Graph Engines. *IEEE Trans. Comput.* 14, 8 (2021), 1–10.
- [11] Leskovec Jure. 2014. SNAP Datasets: Stanford large network dataset collection. Retrieved December 2021 from <http://snap.stanford.edu/data> (2014).
- [12] Arpandee Khatua, Vikram Sharma Mailthody, Bhagyashree Taleka, Tengfei Ma, Xiang Song, and Wen-mei Hwu. 2023. Igb: Addressing the gaps in labeling, features, heterogeneity, and size of public graph datasets for deep learning research. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4284–4295.
- [13] Thomas N Kipf and Max Welling. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- [14] Aapo Kyrola, Guy E Blelloch, and Carlos Guestrin. 2012. GraphChi: Large-Scale Graph Computation on Just a PC. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX, 31–46.
- [15] Jure Leskovec, Jon Kleinberg, and Christos Faloutsos. 2007. Graph evolution: Densification and shrinking diameters. *ACM Transactions on Knowledge Discovery from Data* 1, 1 (2007), 1–41.
- [16] Zekun Li, Zeyu Cui, Shu Wu, Xiaoyu Zhang, and Liang Wang. 2019. Fi-gnn: Modeling feature interactions via graph neural networks for ctr prediction. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*. ACM, 539–548.
- [17] Zhiqi Lin, Cheng Li, Youshan Miao, Yunxin Liu, and Yinlong Xu. 2020. Pagraph: Scaling gnn training on large graphs via computation-aware caching. In *Proceedings of the 11th ACM Symposium on Cloud Computing (SoCC)*. ACM, 401–415.
- [18] Yang Liu, Xiang Ao, Zidi Qin, Jianfeng Chi, Jinghua Feng, Hao Yang, and Qing He. 2021. Pick and choose: a GNN-based imbalanced learning approach for fraud detection. In *Proceedings of the ACM Web Conference 2021 (WWW)*. ACM, 3168–3177.
- [19] Linhao Luo, Yixiang Fang, Xin Cao, Xiaofeng Zhang, and Wenjie Zhang. 2021. Detecting communities from heterogeneous graphs: A context path-based graph neural network model. In *Proceedings of the 30th ACM international conference on information & knowledge management (CIKM)*. ACM, 1170–1180.
- [20] Aditya Pal, Chantat Eksombatchai, Yitong Zhou, Bo Zhao, Charles Rosenberg, and Jure Leskovec. 2020. Pinnersage: Multi-modal user embedding framework for recommendations at pinterest. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*. ACM, 2311–2320.
- [21] Jeongmin Brian Park, Vikram Sharma Mailthody, Zaid Qureshi, and Wen-mei Hwu. 2024. Accelerating Sampling and Aggregation Operations in GNN Frameworks with GPU Initiated Direct Storage Accesses. *Proceedings of the VLDB Endowment* 17, 6 (2024), 1227–1240.
- [22] Yeonhong Park, Sunhong Min, and Jae W Lee. 2022. Ginex: Ssd-enabled billion-scale graph neural network training on a single machine via provably optimal in-memory caching. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2626–2639.
- [23] Masoud Reyhani Hamedani, Jin-Su Ryu, and Sang-Wook Kim. 2023. Geltor: A graph embedding method based on listwise learning to rank. In *Proceedings of the ACM Web Conference 2023 (WWW)*. ACM, 6–16.
- [24] Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. 2019. Droppedge: Towards deep graph convolutional networks on node classification. *arXiv preprint arXiv:1907.10903* (2019).
- [25] Amitabha Roy, Ivo Mihailovic, and Willy Zwaenepoel. 2013. X-Stream: Edge-Centric Graph Processing Using Streaming Partitions. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 472–488.
- [26] Zeang Sheng, Wentao Zhang, Yangyu Tao, and Bin Cui. 2024. OUTRE: An Out-of-Core De-REDundancy GNN Training Framework for Massive Graphs within A Single Machine. *Proceedings of the VLDB Endowment* 17, 11 (2024), 2960–2973.
- [27] Jie Sun, Mo Sun, Zheng Zhang, Jun Xie, Zuo Cheng Shi, Zihan Yang, Jie Zhang, Fei Wu, and Zeke Wang. 2023. Helios: An Efficient Out-of-core GNN Training System on Terabyte-scale Graphs with In-memory Performance. *arXiv preprint arXiv:2310.00837* (2023).
- [28] Petar Velićović, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- [29] Roger Waleffe, Jason Mohoney, Theodoros Rekatsinas, and Shivaram Venkataraman. 2023. Mariusgnn: Resource-efficient out-of-core training of graph neural networks. In *Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys)*. 144–161.
- [30] Minjie Yu Wang. 2019. Deep graph library: Towards efficient and scalable deep learning on graphs. In *ICLR workshop on representation learning on graphs and manifolds*.
- [31] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. 2020. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* 32, 1 (2020), 4–24.
- [32] Yahoo. 2024. Yahoo Webscope dataset. <https://webscope.sandbox.yahoo.com>
- [33] Liangwei Yang, Zhiwei Liu, Yingdong Dou, Jing Ma, and Philip S Yu. 2021. Consisrec: Enhancing gnn for social recommendation via consistent neighbor aggregation. In *Proceedings of the 44th International ACM SIGIR conference on Research and Development in Information Retrieval (SIGIR)*. ACM, 2141–2145.
- [34] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining (KDD)*. ACM, 974–983.
- [35] Seongjun Yun, Seoyoon Kim, Junhyun Lee, Jaewoo Kang, and Hyunwoo J Kim. 2021. Neo-gnns: Neighborhood overlap-aware graph neural networks for link prediction. *Advances in Neural Information Processing Systems* 34 (2021), 13683–13694.
- [36] Muhan Zhang and Yixin Chen. 2018. Link prediction based on graph neural networks. *Advances in Neural Information Processing Systems* 31 (2018).
- [37] Shichang Zhang, Jiani Zhang, Xiang Song, Soji Adeshina, Da Zheng, Christos Faloutsos, and Yizhou Sun. 2023. PaGE-Link: Path-based graph neural network explanation for heterogeneous link prediction. In *Proceedings of the ACM Web Conference 2023 (WWW)*. ACM, 3784–3793.
- [38] Weitong Zhang, Ronghua Shang, Zhiyuan Li, Rui Sun, and Jun Du. 2023. Personalized web page ranking based graph convolutional network for community detection in attribute networks. *IEEE Access* (2023).
- [39] Ziwei Zhang, Peng Cui, and Wenwu Zhu. 2020. Deep learning on graphs: A survey. *IEEE Transactions on Knowledge and Data Engineering* 34, 1 (2020), 249–270.
- [40] Da Zheng, Chao Ma, Minjie Wang, Jinjing Zhou, Qidong Su, Xiang Song, Quan Gan, Zheng Zhang, and George Karypis. 2020. DistDGL: Distributed graph neural network training for billion-scale graphs. In *2020 IEEE/ACM 10th Workshop on Irregular Applications: Architectures and Algorithms (IA3)*. IEEE, 36–44.
- [41] Da Zheng, Disa Mhembere, Randal Burns, Joshua Vogelstein, Carey E Priebe, and Alexander S Szalay. 2015. FlashGraph: Processing Billion-Node Graphs on an Array of Commodity SSDs. In *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*. USENIX, 45–58.
- [42] Da Zheng, Xiang Song, Chengru Yang, Dominique LaSalle, and George Karypis. 2022. Distributed hybrid cpu and gpu training for graph neural networks on billion-scale heterogeneous graphs. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4582–4591.
- [43] Fan Zhou, Chengtai Cao, Kunpeng Zhang, Goce Trajcevski, Ting Zhong, and Ji Geng. 2019. Meta-gnn: On few-shot node classification in graph meta-learning. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*. ACM, 2357–2360.
- [44] Rong Zhu, Kun Zhao, Hongxia Yang, Wei Lin, Chang Zhou, Baole Ai, Yong Li, and Jingren Zhou. 2019. AliGraph: A Comprehensive Graph Neural Network Platform. *Proceedings of the VLDB Endowment* 12, 12 (2019), 2094–2105.